Training Logs and Summary

IE 7615 Deep Learning for AI · Project 2, Milestone 2 · Group 02: Abdellah Faleh, Jin-Woo Hong, Samuel Tong, Tristan Lyons · Logs: *m2_integration/* and *m2_generation/*

1a. BERT → CLIP adapter training

A BERT → CLIP adapter (17.0M parameters) learned to turn BERT embeddings into the 77×768 CLIP embeddings the U-Net reads. Train: 7,613 captions of the 1,600 train photos. Validation: 200 val captions. Loss: mean squared error on standardized values (a model that always outputs the average scores 1.00). AdamW, learning rate 3×10^{-4} (one-cycle), batch 64, 15 epochs = 1,785 steps, seed 42.

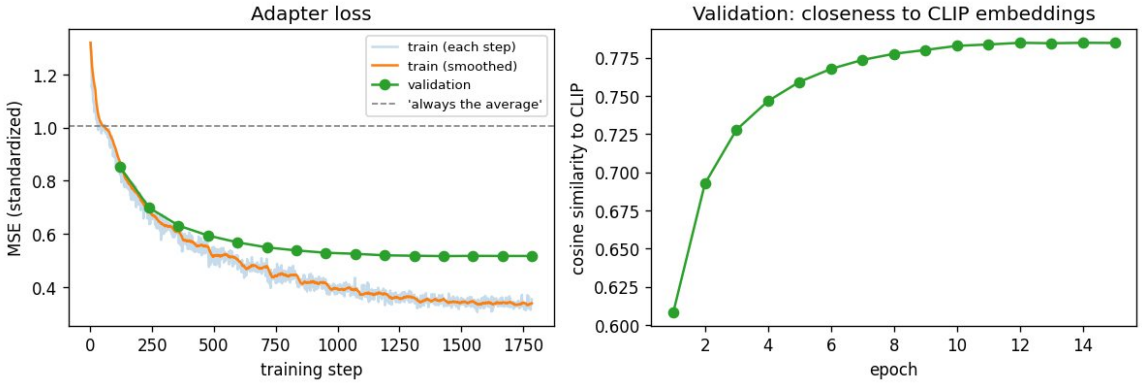


Figure 1. Loss at every step (light), smoothed, and after every epoch on validation (left); closeness to CLIP's embeddings on validation (right). Log files: adapter_train_steps.csv, adapter_train_epochs.csv.

Epoch	1	3	5	8	10	12 (best)	15
Train loss	0.991	0.633	0.515	0.417	0.375	0.350	0.337
Validation loss	0.853	0.631	0.568	0.529	0.519	**0.517**	0.517
Validation cosine to CLIP	0.608	0.728	0.759	0.778	0.783	0.785	0.785

Table 1. Selected epochs.

Stability: the loss fell smoothly with no spikes (gradient clipping at 1.0). The validation loss stopped improving after about epoch 10 while the training loss kept falling: the gap (0.34 vs. 0.52) shows mild overfitting, so early stopping at about 10 epochs would save a third of the time with the same result.

1b. LoRA fine-tuning of the U-Net (notebook 04_sd_lora_finetuning)

Rank-8 LoRA on the U-Net attention layers: **1,594,368 of 861,115,332** U-Net parameters trained (0.19%). 1,600 train photo–caption pairs, 800 steps, batch size 1, learning rate 10^{-4}, noise-prediction MSE, frozen CLIP and VAE, Tesla T4. The loss (mean of every 25 steps, log: *lora_loss_log.csv*) stayed between **0.10 and 0.24** (mean 0.175); first 200 steps 0.192, last 200 steps 0.185, so **no clear drop**. With batch size 1 each step sees one random noise level, so the loss is very noisy; a longer run with a larger batch is needed to see a trend.

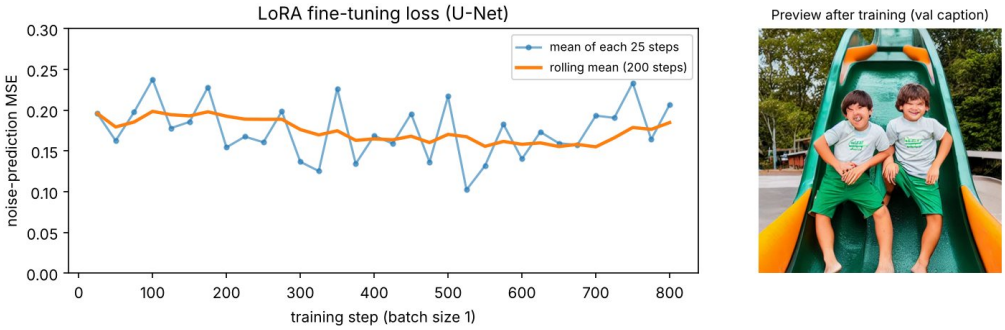


Figure 2. LoRA loss (left) and a validation-caption preview after training (right): realistic, but two boys instead of three and the shirt and pant colors are mixed up.

2. Runtime and GPU memory (Tesla T4, 15 GB)

Measure	Value
Software	Python 3.13, PyTorch 2.11, Diffusers 0.42.1, Transformers 5.22
Load the 5 model parts / compute BERT and CLIP features for 7,813 captions	57.8 s / 29.6 s
Adapter training: total / per epoch	307 s (5.1 min) / 20.5 s
Peak GPU memory: adapter training	6.35 GB
Peak GPU memory: generating one image (notebook 04 / 05 with the CLIP scorer loaded)	2.8 GB / 4.3 GB
Time per image: baseline (PNDM, 30 steps, guidance 7.5)	5.0 s (range 4.4–5.8)
Time per denoising step (all 5 samplers)	about 0.19 s; 10 steps = 2.0 s, 50 steps = 9.0 s
Guidance scale 1 (one U-Net pass per step instead of two)	3.4 s vs. 5.6 s per image
Total run time: notebook 04 / notebook 05 (202 images)	10.3 min / 20.9 min

Table 2. Runtime metrics (from summary.json and results.csv).

3. Sampler diagnostics (inference)

For the baseline image we logged every step (sampler_diagnostics.csv). The latent's spread falls from 1.00 to 0.67 as noise is removed. The text signal (the difference between the U-Net's prediction with and without the caption) stays about 1–2 across the steps, with one peak of 3.7 at step 10, when the layout forms. PNDM uses 31 U-Net calls for 30 steps (one extra warm-up step).

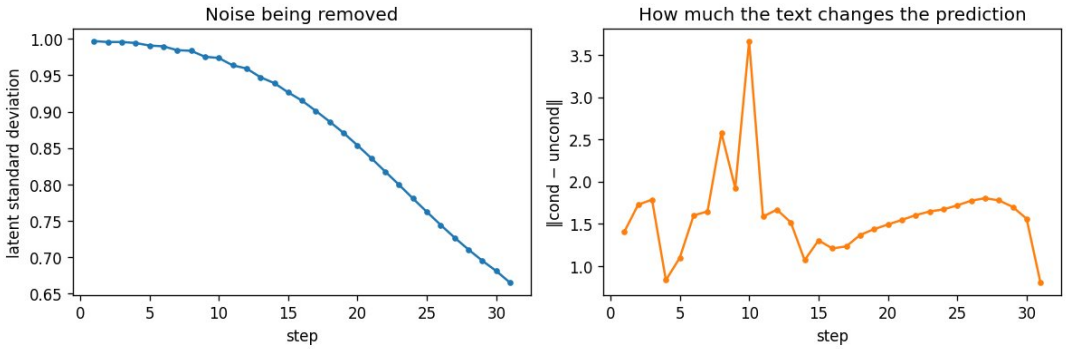


Figure 3. Latent spread and text signal at each step (PNDM, 30 steps, guidance 7.5).

4. Parameter-tuning observations

- **Guidance scale** (1, 3, 5, 7.5, 10, 15): prompt match rises from 20.8 to 26.0 by 7.5 and then stops; color saturation keeps rising (31 → 58) and variety between seeds falls (similarity 68 → 90). Best range: 5–7.5.
- **Sampler and steps:** all samplers cost the same per step. DPM-Solver++ and Euler settle by 20 steps; PNDM and DDIM break or blur at 10 steps; DDPM gives a new picture at every step count. DPM-Solver++ at 20 steps saves a third of the time of our baseline.
- **Noise schedule:** using a schedule the model was not trained with hurts: cosine gives blurry images and linear over-saturates. Karras sigmas changed almost nothing at 20 steps.
- **CLIP score** moved only 25.5–27.4 across all samplers, even for broken images, so we also used the pixel difference to each sampler's 50-step image and looked at every grid.

5. Lessons learned

What worked	What did not	What we would change next
Our own loop reproduces the pipeline exactly (0.0 difference); the text controls the image (6 of 6); the adapter learned fast (5 min) and makes BERT usable (21.4 vs. 9.9); LoRA fine-tuning fits on a free T4.	Rare words get lost with BERT + adapter (glow sticks, roller coaster); guidance cannot fix counting or numbers; one crash from a 0/1 mask instead of a True/False mask; the LoRA loss did not clearly drop in 800 steps.	Stop adapter training at about 10 epochs and train on more captions; use DPM-Solver++ with 20 steps for large runs; run the LoRA held-out comparison and a longer LoRA run; add FID and Inception Score in Milestone 3.